ACADEMIC CAPSTONE DEFENSE REVIEWER

GRAMMATICA.SITE

Comprehensive System Architecture, Technical Implementation Manual, and Complete Source Code Reviewer for Capstone Mock Defense

Project Title:	Grammatica: An Intelligent Multi-Modal Language Learning and Mentorship Platform
Primary Technology Stack:	Flutter 3.x (Web & Android APK), Firebase Firestore, Firebase Auth, FastAPI Backend (Distributed Idempotency Engine), Google Gemini 2.5 Flash, Agora RTC, PayMongo API (QR Ph & Idempotent Checkout), Brevo Transactional Email, Vosk Acoustic Engine
Target Review:	Mock Defense & Final Oral Defense Preparation
Document Scope:	Architectural Principles, Subsystem Implementations, Theoretical Algorithms, Distributed Idempotency Engine, File-by-File Catalog (85+ Dart files), Panelist Q&A
Version & Date:	Version 2.5.0 • Academic Year 2026

Developed for rigorous evaluation by the Capstone Defense Panel. Contains comprehensive explanations of algorithms, database structures, security protocols, and integration pipelines.

Table of Contents

1.0 Executive Summary & Theoretical Foundations

2.0 End-to-End System Architecture & High-Level Data Flow

3.0 Core Subsystems & Technical Implementation Deep-Dive

- 3.1 Authentication, Brute-Force Defense & Session Security
- 3.2 Role-Based Access Control (RBAC) & Governance Pipeline
- 3.3 AI Educational Notebook & Multi-Modal Source Ingestion
- 3.4 Cognitive Retention & Spaced Repetition (SuperMemo SM-2)
- 3.5 Interactive Gamified Learning & Acoustic Speech Evaluation
- 3.6 Real-Time Video Conferencing & Mentorship (Agora RTC)
- 3.7 Monetization, PayMongo Dynamic Checkout & Email Receipts
- 3.8 Distributed Idempotency Architecture & Financial Double-Charge Prevention
- 3.9 Content Moderation, Verification & Mandatory Denial Auditing

4.0 Exhaustive File-by-File Catalog (Every Single .dart File)

- 4.1 Configuration & Theme Layer (`lib/config/` , `lib/theme/`)
- 4.2 Domain Models & Data Structures (`lib/models/`)
- 4.3 Central Services & Repositories (`lib/services/`)
- 4.4 UI Presentation Pages & Workspaces (`lib/pages/`)
- 4.5 Educational Notebook Workspaces (`lib/pages/notebook/`)
- 4.6 Administration & Verification Tabs (`lib/pages/admin/`)
- 4.7 Reusable UI Components & Widgets (`lib/widgets/`)
- 4.8 Utilities & Platform Helpers (`lib/utils/`)

5.0 Microservices AI Backend & LLM Fine-Tuning (FastAPI & LoRA)

6.0 Anticipated Defense Questions & Panel Defense Guide (Top 27 Q&A)

1.0 Executive Summary & Theoretical Foundations

Grammatica is a state-of-the-art, cross-platform language learning ecosystem designed to overcome the critical limitations of contemporary language acquisition platforms. Conventional platforms predominantly deliver static, repetitive drills that lack pedagogical contextualization, fail to support multimodal learning materials, and disconnect asynchronous practice from real-time certified educator mentorship.

To address this challenge, Grammatica harmonizes three pillars of cognitive science and software engineering:

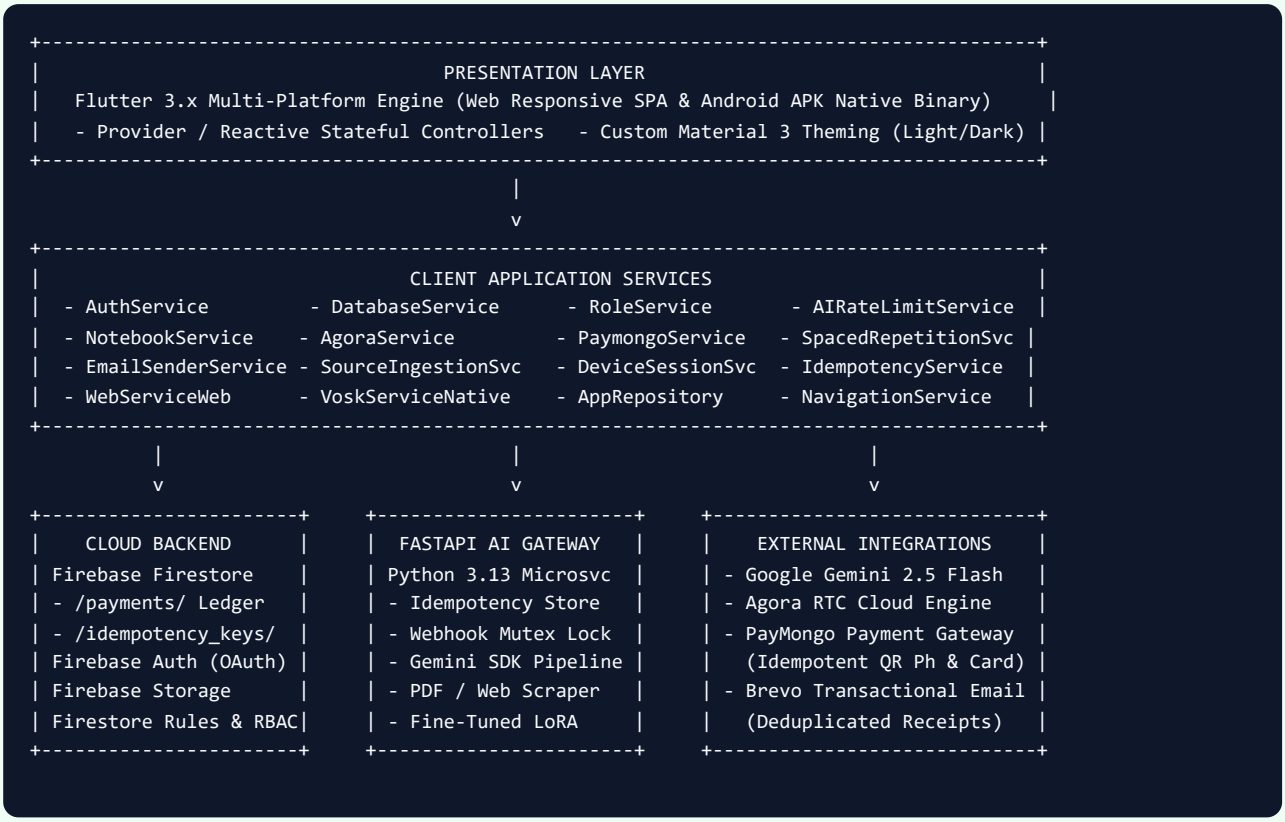
1. **Multimodal Knowledge Ingestion & Contextual AI Synthesis:** Instead of relying on generic pre-canned sentences, educators and learners can ingest real-world curricular materials (PDF documents, syllabus chapters, scholarly web pages, and raw text notes). Powered by the Google Gemini 2.5 Flash large language model, Grammatica parses and transforms these unstructured documents into high-retention pedagogical artifacts: structured interactive lessons, formative assessments, flashcard decks, hierarchical mind maps, chronological timelines, and concise briefing documents.
2. **Empirical Spaced Repetition (The SuperMemo SM-2 Algorithm):** Based on Hermann Ebbinghaus's Forgetting Curve theory, Grammatica incorporates an algorithmic spaced repetition engine. Card review intervals and ease factors dynamically adjust based on individual recall quality scores, ensuring durable memory consolidation with minimal study fatigue.
3. **Synchronous Human Mentorship & Secure Monetization:** Grammatica integrates low-latency, WebRTC-powered video conferencing through the Agora RTC Engine, alongside secure payment processing via the PayMongo API (supporting GCash, Maya, GrabPay, and credit/debit cards). Educators can monetize their expertise through customizable subscription tiers and scheduled 1-on-1 mentorship sessions, complete with automated transactional email receipts.

> **Key Defense Talking Point: The Architectural Problem Statement**
>
> Traditional apps suffer from "context detachment." Grammatica provides "curriculum-grounded learning"—the AI acts as an instructional designer, translating authentic educational material into active retrieval practice while certified educators provide live accountability and guidance.

2.0 End-to-End System Architecture

Grammatica is engineered using a robust, decoupled, and highly responsive **Service-Oriented Architecture (SOA)** paired with the **Repository Pattern** on the Flutter client and asynchronous microservices on the Python backend.

Architectural Topology



2.1 Client-Side Architectural Principles

- **Single Source of Truth:** `DatabaseService` acts as the canonical data gatekeeper, exposing continuous, real-time reactive streams (`StreamBuilder`) from Cloud Firestore. Data mutations are optimistically reflected in the UI while ensuring atomic cloud persistence.
- **Cross-Platform Conditional Compilation:** The codebase leverages conditional imports (e.g., `web_service_web.dart` vs `web_service_stub.dart` , and `vosk_service_native.dart` vs `vosk_service_stub.dart`). This enables native Android builds to execute high-performance C++ speech binaries (Vosk offline acoustic model) while web builds gracefully adapt to browser standards and CORS proxy relays without crashing.
- **Distributed Idempotency & Financial Safety:** End-to-end multi-tier deduplication protects against double billing, duplicate bookings, and race conditions across client (single-flight coalescing), gateway (SHA-256 fingerprinting & 409 Conflict locks), and database (check-and-set Firestore ledgers) layers.
- **Strict Defensive Input Validation:** Every single required user entry point is enforced both on the UI layer (displaying visual asterisk `*` indicators and inline regex validators) and within service invocation guards.

3.0 Core Subsystems & Technical Implementation

3.1 Authentication, Brute-Force Defense & Session Security

Authentication is managed via `AuthService` and backed by Firebase Authentication, supporting traditional credential login, federated Google OAuth, and Phone OTP verification.

Brute-Force Attack Mitigation (The 3-Attempt Lockout System)

To prevent credential stuffing and brute-force password guessing, Grammatica implements a sophisticated local-and-cloud session gatekeeper encapsulated in `DeviceSessionService`:

- **Failure Tracking:** Every failed login attempt increments a device-session counter stored in encrypted local preferences (`SharedPreferences`).
- **Lockout Threshold:** Upon reaching **3 consecutive failed attempts**, the device is immediately placed in a strict **5-minute lockout state**. All authentication buttons are disabled, and a persistent visual countdown timer is presented to the user.
- **Sliding Reset Window:** If a user fails once or twice but does not attempt another login for **15 minutes**, the failed attempts counter automatically resets to zero, preventing false-positive lockouts for legitimate users over long intervals.
- **Successful Authentication:** Any verified login instantly wipes all failure counters and lockout timestamps.

Sensitive Operation Re-Authentication Protocol

In compliance with cybersecurity best practices, high-risk actions—such as modifying primary display names, resetting passwords, changing registered phone numbers, or executing account deletions—cannot be triggered silently. Grammatica prompts an obligatory modal (`_ReauthDialog`) requiring the user to re-input their existing credentials to obtain a fresh Firebase Auth Token before mutating the database.

3.2 Role-Based Access Control (RBAC) & Governance Pipeline

Grammatica enforces a rigorous 4-tier RBAC architecture defined in `RoleService`:

Role	Permissions & System Capabilities	AI Daily Quota
LEARNER	Access public & subscribed lessons, take practice quizzes, participate in Spelling Bee & Pronunciation quizzes, book live mentorship, maintain personal study notebooks, manage subscriptions.	**20 requests / day**
EDUCATOR	All Learner privileges + Author custom lessons & quizzes, schedule mentorship sessions via Agora, configure Standard/Premium subscription tiers, upload multi-modal syllabus sources, review subscriber rosters.	**50 requests / day**
VALIDATOR	Audit public content queue, review published lessons & quizzes for grammatical correctness and community standards, approve or deny content with mandatory audit logging.	**100 requests / day**
ADMIN	Global superuser: Manage user roles, revoke permissions, configure landing page settings, inspect comprehensive system telemetry, broadcast notifications, bypass quota limits.	**Unlimited (∞)**

Role Elevation Workflow: Users register by default as Learners. To become an Educator or Validator, users must submit an official application through `RoleApplicationPage`, uploading credentials, institutional affiliations, and government-issued IDs. These submissions are vetted by Validators and finalized by Administrators.

3.3 AI Educational Notebook & Multi-Modal Source Ingestion

The **Educator's AI Notebook** represents Grammatica's core technical innovation. Rather than relying on ungrounded generative prompts that frequently hallucinate, Grammatica implements a **Grounding-First Pipeline**:

1. **Source Ingestion (`SourceIngestionService`):**
   - **PDF Ingestion:** Extracts textual streams directly from uploaded academic documents and lecture slide decks.
   - **Web Page Ingestion:** Fetches external articles, grammar guides, or encyclopedia links, stripping extraneous HTML boilerplates and script tags. Web clients utilize an intelligent CORS relay to ensure seamless browser parsing.
   - **Direct Text Notes:** Allows educators to paste custom curriculum outlines, lecture transcripts, or vocabulary lists.
2. **Structured Prompt Engineering (`notebook_prompts.dart` & `ai_logic_service.dart`):** Raw text chunks are wrapped in specialized system instructions instructing Gemini 2.5 Flash to generate deterministic, syntactically valid JSON structures.
3. **Pedagogical Artifact Synthesis:** From a single source, the engine can synthesize:
   - **Interactive Lessons:** Markdown-formatted explanatory articles featuring headings, bold grammar concepts, illustrative image blocks, and companion quiz linkages.
   - **Formative Quizzes:** Multi-format questions (Multiple Choice with distractor explanations, Fill-in-the-Blank, Direct Identification, Reading Passages, and Image Questions).
   - **Flashcard Decks:** Front/back vocabulary cards paired with phonetics and example sentences.
   - **Study Guides, Mind Maps, Timelines, & FAQs:** Structured visual and textual artifacts catering to diverse learning modalities.
4. **Double-Click Prevention & Quota Guards:** To prevent network congestion and redundant token consumption, all AI generation triggers are wrapped in `GenerationLoadingOverlay`. The UI disables triggering buttons and displays rotating pedagogical progress indicators until the transaction fully resolves.

3.4 Cognitive Retention & Spaced Repetition (SuperMemo SM-2)

Flashcards in Grammatica are not simply shuffled randomly. They are scheduled according to the celebrated **SuperMemo SM-2 Spaced Repetition Algorithm**, mathematically modeled inside `SpacedRepetitionService`.

MATHEMATICAL FORMULATION OF THE SM-2 ENGINE

For each review session, the user evaluates their recall quality $q \in \{0, 1, 2, 3, 4, 5\}$:

- $q = 5$: Perfect response, zero hesitation.
- $q = 4$: Correct response after slight hesitation.
- $q = 3$: Correct response recalled with serious difficulty.
- $q = 2$: Incorrect response; where the correct answer seemed easy to recall.
- $q = 1$: Incorrect response; familiar with the word but failed to recall.
- $q = 0$: Complete blackout.

1. Ease Factor (EF) Calculation:

```
EF' = EF + (0.1 - (5 - q) * (0.08 + (5 - q) * 0.02))
```

Constraint: $EF' \ge 1.30$. If EF' drops below 1.30, it is clamped to 1.30 to prevent perpetual scheduling stagnation. Default initial $EF = 2.50$.

2. Inter-Repetition Interval (I) Calculation:

```
If q < 3 (Recall Failure):
    Repetitions (n) = 0
    Next Interval I(1) = 1 day

If q >= 3 (Recall Success):
    If n == 0: I(1) = 1 day
    If n == 1: I(2) = 6 days
    If n >= 2: I(n) = I(n-1) * EF'
    Repetitions (n) = n + 1
```

Firestore Persistence: Each user's card metadata (`easeFactor` , `interval` , `repetitions` , `nextReviewDate` , `reviewCount`) is persisted atomically in Firestore. When opening `LearnerDecksPage` , Grammatica queries cards where `nextReviewDate <= DateTime.now()` , presenting learners with precisely calibrated daily review queues.

3.5 Interactive Gamified Learning & Acoustic Speech Evaluation

Spelling Bee Game Engine (`SpellingBeePage`)

The Spelling Bee module presents learners with auditory vocabulary challenges. The word is played using speech synthesis while contextual definitions and part-of-speech hints are rendered. Learners enter their answer letter-by-letter with instant reactive feedback, streak multipliers, sound effects, and confetti celebrations upon completion. Words are organized by difficulty (`easy` , `medium` , `hard`) managed through the Admin Spelling Words tab.

Acoustic Pronunciation Quiz Engine (`PronunciationQuizPage` & `VoskService`)

Pronunciation practice requires evaluating spoken audio against phonetic standards. Grammatica employs a hybrid architecture:

- **Mobile Native Android APK:** Integrates an offline, privacy-first **Vosk Acoustic Model** via dynamic C++ platform channels (`vosk_service_native.dart`). Audio waveforms recorded from the device microphone are processed locally

into phoneme sequences, calculating a percentage match score without sending voice recordings to external servers.

- **Web Application SPA:** Seamlessly falls back to the Web Speech Recognition API (`vosk_service_stub.dart`) to guarantee zero setup barriers on modern web browsers.

3.6 Real-Time Video Conferencing & Mentorship (Agora RTC)

Mentorship sessions take place within Grammatica's native video conferencing portal (`ConferencePage`), built upon the **Agora RTC SDK**:

- **Token Generation Pipeline:** When a learner or educator joins a conference room, `AgoraService` requests or dynamically calculates an authorized HMAC-SHA256 RTC token using the app's secure credentials, ensuring unauthorized third parties cannot eavesdrop on private sessions.
- **Cross-Platform Media Engine:** The client establishes an ultra-low latency WebRTC stream. Native mobile builds render hardware-accelerated video frames via Android SurfaceViews, while web clients mount dynamic HTML5 video elements.
- **Concurrent Real-Time Chat:** Participants can toggle microphones, flip cameras, and communicate via a synchronized side chat stream backed by Firestore real-time snapshots.

3.7 Monetization, PayMongo Dynamic Checkout & Email Receipts

Grammatica enables educators to commercialize their pedagogical work through subscription tiers:

- **Tier Structure:** *Standard Tier* (Up to $10/mo - grants access to exclusive educator lessons and quiz folders) and *Premium Tier* (Up to $30/mo - includes 1-on-1 scheduled live Agora mentorship sessions).
- **PayMongo API Integration (`PayMongoService`):**
  - **Dynamic Hosted Checkout:** Generates secure PayMongo checkout sessions supporting credit/debit cards, GCash, Maya, and GrabPay. All link generation calls attach cryptographic `Idempotency-Key` headers and cache active URLs in-memory to prevent duplicate checkout creations.
  - **Direct QR Ph Code Fallback:** Generates real-time, high-contrast QR Ph payment strings displayed in a dedicated modal (`PaymongoQrDialog`). Users scan the QR directly in banking apps. A single-shot mutex (`_hasDispatchedSuccess`) prevents race conditions between polling timers and manual triggers.
- **Automated Transactional Email Receipts (`EmailSenderService`):** Upon successful payment confirmation, the system generates an itemized, responsive HTML receipt and dispatches it directly to the customer's email via the **Brevo API** (Sendinblue), complete with transaction ID, tier details, and renewal dates. A local Dart email relay (`scripts/local_email_relay.dart`) serves as a zero-downtime offline fallback during local testing. Duplicate receipt dispatch is strictly blocked by the idempotency ledger.

3.8 Distributed Idempotency Architecture & Financial Double-Charge Prevention

In financial and distributed systems, network unreliability and rapid user interactions can lead to severe side-effects such as double billing, duplicate session bookings, and redundant AI credit deductions. To guarantee absolute consistency, Grammatica implements a cohesive **3-Tier Distributed Idempotency Architecture** spanning client, gateway, and database layers:

The Three Pillars of Grammatica Idempotency

1. **Client-Tier Single-Flight Coalescing (`IdempotencyService`):** Rapid button taps within short windows join the exact same active `Future` , completely preventing multiple concurrent HTTP requests from ever leaving the client device. Deterministic RFC 4122 v4 keys (`pay_link_{studentId}_{educatorId}_{timestampMinutes}`) guarantee replay safety.
2. **Gateway Middleware Fingerprinting (`IdempotencyMiddleware` & `IdempotencyStore`):** Mutating HTTP routes (`POST` , `PUT` , `PATCH` , `DELETE`) are locked via SHA-256 payload fingerprints. Concurrent requests return `HTTP 409 Conflict` (with `Retry-After: 2`); modified payloads under the same key reject with `HTTP 422 Unprocessable Entity` ; and repeated identical calls replay the cached response with `Idempotent-Replay: true` .
3. **Database Transactional Check-and-Set Ledger (`DatabaseService`):** Transactions in `/payments/{paymentId}` use deterministic doc IDs (`pay_{linkId}`). Existing `paid` records return `PaymentRecordResult(isDuplicate: true)` , safely skipping duplicate subscription increments and redundant email receipts.

PayMongo Webhook Deduplication: Inbound webhooks from payment gateways operate on an *at-least-once delivery* model. The FastAPI webhook route locks each event ID (`evt_...`) via `IdempotencyStore` . Whichever path resolves first —the background webhook or the client's foreground polling loop—commits the payment atomically. The secondary process recognizes the existing record, flags it as duplicate, and quietly exits without side-effects.

3.9 Content Moderation, Verification & Mandatory Denial Auditing

To ensure curricular excellence, educator-published lessons and quizzes enter a pending verification queue reviewed by certified Validators (`ValidatorDashboardTab`).

The Mandatory Denial Explanation Protocol (`ContentDenialDialog`)

To prevent arbitrary or biased censorship, a Validator **cannot** reject content with a single click. If a Validator clicks "Deny", the system forces open `ContentDenialDialog` , which requires a **minimum 30-character comprehensive explanation** citing pedagogical shortcomings or guideline infractions. This audit log is permanently stored in Firestore and dispatched to the educator to facilitate constructive revision.

4.0 Exhaustive File-by-File Codebase Catalog

Below is an exhaustive architectural catalog detailing every relevant `.dart` source file across the Grammatica project, explaining its exact responsibility, key classes, and design decisions.

4.1 Configuration & Theme Layer (`lib/config/` , `lib/theme/`)

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/config/ai_config.dart`	`AIConfig`	Defines constants for AI generation: default model identifier (`gemini-2.5-flash`), temperature settings, topP, topK, maximum token limits, and safety category thresholds to prevent toxic output.
`lib/config/notebook_prompts.dart`	`NotebookPrompts`	Contains strictly structured system prompts for generating lessons, quizzes, flashcards, mind maps, study guides, timelines, briefings, and FAQs from raw educational sources.
`lib/theme/app_colors.dart`	`AppColors`	Central palette design system: Primary Green (`#81B655`), Deep Slate, Accent Purples, Dark Mode Backgrounds (`#1A1D1A`), semantic feedback greens/reds, and card elevation borders.
`lib/theme/app_theme.dart`	`AppTheme`	Implements Material 3 `ThemeData` for both Light and Dark modes. Configures Google Fonts (Outfit for headings, Inter for body copy), input decoration themes, elevated button styles, and card themes.
`lib/firebase_options.dart`	`DefaultFirebaseOptions`	Auto-generated Firebase CLI configuration providing platform-specific API keys, Project IDs, Storage Buckets, and App IDs for Android, Web, and iOS deployments.
`lib/main.dart`	`GrammaticaApp` , `main()`	Application entry point. Initializes Flutter bindings, loads environment variables (`.env`), initializes Firebase, configures navigation observers, and boots the initial authentication state router.

4.2 Domain Models & Data Structures (`lib/models/`)

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/models/ai_models.dart`	`AIQuizResponse` , `AIQuestion` , `AILessonResponse` , `ContentBlock`	JSON-serializable data models mapping structured responses from Google Gemini into typed Flutter objects. Includes helper methods for Markdown conversion and image block binding.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/models/content_visibility.dart`	`ContentVisibility` (Enum)	Defines content visibility tiers: `public` (free for all), `subscribersOnly` (requires active educator subscription), and `private` (draft / personal).
`lib/models/flashcard_models.dart`	`FlashcardDeck`, `FlashcardItem`	Represents flashcard collections. Contains SuperMemo SM-2 state fields: `easeFactor`, `intervalDays`, `repetitionNumber`, `nextReviewDate`, and mastery percentages.
`lib/models/notebook_models.dart`	`Notebook`, `NotebookSource`, `NotebookOutput`	Core models for the AI Notebook workspace. Tracks ingested source documents (PDF, URL, Text) and synthesized outputs (Lesson, Quiz, MindMap, Timeline, FAQ, Briefing, StudyGuide).
`lib/models/notification.dart`	`AppNotification`	Data model for in-app user notifications (e.g., mentorship confirmations, payment receipts, validation status changes, content approval/denial alerts).
`lib/models/published_content_item.dart`	`PublishedContentItem`	Represents items submitted by educators to the public repository. Tracks verification status (`pending`, `approved`, `denied`), denial rationale, and validator UID.
`lib/models/payment_models.dart` (and `DatabaseService`)	`PaymentRecordResult`	Result object returned from payment ledger mutations. Encapsulates `isSuccess`, `isDuplicate`, `paymentId`, and human-readable audit messages to safely gate downstream receipt dispatch.
`lib/models/spelling_word.dart`	`SpellingWord`, `SpellingDifficulty`	Defines vocabulary items for the Spelling Bee game: word text, phonetics, audio URL, definition, example sentence, and difficulty tier.
`lib/models/study_guide_models.dart`	`StudyGuide`, `MindMapNode`, `TimelineEvent`	Hierarchical tree data structures for mind maps, chronologically ordered timeline nodes, and sectioned study guides.

4.3 Central Services & Repositories (`lib/services/`)

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/services/auth_service.dart`	`AuthService`	Handles user authentication lifecycle: email/password registration, login, logout, password reset emails, Google OAuth, sensitive operation re-authentication, and account deletion.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/services/database_service.dart`	`DatabaseService`	Massive (100KB) data access singleton. Manages all Cloud Firestore collections: user profiles, lesson CRUD, quiz CRUD, assessments, spelling words, mentorship sessions, reviews, denial audits, and the idempotent `PaymentLedger` (/payments/ & /idempotency_keys/) with check-and-set duplicate protection.
`lib/services/idempotency_service.dart`	`IdempotencyService`	Client-side idempotency coordinator. Generates cryptographic RFC 4122 v4 UUIDs, deterministic payment/mentorship/AI keys, coalesces in-flight requests (single-flight deduplication), and persists cached results in local storage with TTL.
`lib/services/role_service.dart`	`RoleService` , `UserRole`	Enforces Role-Based Access Control (RBAC). Evaluates role permissions, processes educator/validator applications, updates roles, and exposes real-time role streams.
`lib/services/ai_logic_service.dart`	`AILogicService`	Direct low-level client for Google Gemini 2.5 Flash. Assembles prompt templates, enforces structured JSON response schemas, injects `Idempotency-Key` headers, handles API error parsing, and manages token usage.
`lib/services/ai_generation_service.dart`	`AIGenerationService`	High-level orchestration service that coordinates source text extraction, quota validation, prompt dispatch, and model mapping for lessons, quizzes, and study guides with deterministic request keys.
`lib/services/ai_rate_limit_service.dart`	`AIRateLimitService`	Enforces daily AI request quotas based on user roles. Increments usage counters in Firestore, calculates remaining quotas, and triggers reset checks at midnight UTC.
`lib/services/notebook_service.dart`	`NotebookService`	Manages educator notebooks, source attachment, artifact synthesis persistence, citation linking, and direct export to the public publishing pipeline.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/services/source_ingestion_service.dart`	`SourceIngestionService`	Handles multi-modal file ingestion. Extracts clean text from uploaded PDFs, scrapes web articles via CORS-friendly relays, and normalizes plain text lecture notes.
`lib/services/spaced_repetition_service.dart`	`SpacedRepetitionService`	Implements the SuperMemo SM-2 algorithm. Calculates next review intervals, ease factor modifications, updates flashcard review queues, and records recall session histories.
`lib/services/paymongo_service.dart`	`PayMongoService`	Interacts with the PayMongo API. Creates dynamic checkout sessions (Cards, GCash, Maya) with injected `Idempotency-Key` headers, caches active payment links in-memory, generates high-contrast QR Ph codes, and coordinates single-shot UI locks.
`lib/services/email_sender_service.dart`	`EmailSenderService`	Transactional email dispatcher. Transmits responsive HTML payment receipts and subscription renewal notifications via Brevo API, with local Dart relay fallback.
`lib/services/agora_service.dart`	`AgoraService`	Manages Agora RTC Engine initialization, channel joining, dynamic token generation, audio/video device toggling, and multi-user stream event handling.
`lib/services/device_session_service.dart`	`DeviceSessionService`	Security gatekeeper managing the 3-attempt brute-force protection. Enforces 5-minute lockout windows, 15-minute sliding resets, and persists device session fingerprints.
`lib/services/notification_service.dart`	`NotificationService`	Creates in-app notifications, marks alerts as read/unread, and exposes live Firestore notification streams to the UI.
`lib/services/navigation_service.dart`	`NavigationService`	Provides global navigator keys and helper methods for contextless navigation, route pushes, and deep link resolution.
`lib/services/vosk_service.dart` (and `_native`, `_stub`)	`VoskService`	Cross-platform abstraction for speech recognition. Executes offline Vosk acoustic models on native Android APK while stubbing for web standards.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/services/web_service.dart` (and `_web`, `_stub`)	`WebService`	Platform-aware HTTP client with intelligent CORS proxy handling for web single-page app builds.
`lib/services/app_repository.dart`	`AppRepository`	In-memory caching coordinator reducing repetitive network trips for static assets, category lists, and user preferences.

4.4 UI Presentation Pages & Workspaces (`lib/pages/`)

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/pages/landing_page.dart`	`LandingPage`	Public showcase landing page. Features hero banners, live preview cards, dynamic pricing matrices, testimonials, and direct onboarding call-to-actions.
`lib/pages/home_page.dart`	`HomePage`	Learner dashboard. Displays daily learning streaks, recent progress cards, quick action shortcuts, recommended lessons, and active mentorship sessions.
`lib/pages/login_page.dart`	`LoginPage`	User sign-in portal. Implements required field asterisks (`Email Address *`, `Password *`), 3-attempt lockout countdown, and Google OAuth trigger.
`lib/pages/signup_page.dart`	`SignupPage`	3-Step registration wizard with real-time email availability validation, phone number formatting, password strength scoring, and terms of service modal.
`lib/pages/forgot_password_page.dart`	`ForgotPasswordPage`	Password reset interface. Dispatches official Firebase recovery links to verified emails with real-time regex validation (`Recovery Email *`).
`lib/pages/otp_verification_page.dart`	`OtpVerificationPage`	6-Digit SMS / Phone verification screen confirming user mobile identity during registration.
`lib/pages/onboarding_page.dart`	`OnboardingPage`	Introductory carousel welcoming newly registered learners and demonstrating core features (Notebook, Mentorship, Games).

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/pages/profile_page.dart`	`ProfilePage`	User profile manager. Allows updating usernames, phone numbers, bios, avatars, re-authenticating sensitive changes, and initiating account deletion.
`lib/pages/settings_page.dart`	`SettingsPage`	User preferences portal. Controls Light/Dark mode toggling, audio playback settings, notification preferences, and legal disclaimers.
`lib/pages/practice_tab.dart`	`PracticeTab`	Interactive practice hub offering entry points into Spelling Bee, Pronunciation Quizzes, and Formative Assessments.
`lib/pages/spelling_bee_page.dart`	`SpellingBeePage`	Gamified spelling competition. Plays audio pronunciations, provides phonetic and definition hints, tracks streaks, and validates letter entries.
`lib/pages/pronunciation_quiz_page.dart`	`PronunciationQuizPage`	Speech assessment interface. Records learner vocalization and compares phoneme sequences against targets using the Vosk acoustic model.
`lib/pages/quizzes_page.dart`	`QuizzesPage`	Searchable quiz catalog categorized by grammar topic, difficulty level, and completion status.
`lib/pages/quiz_detail_page.dart`	`QuizDetailPage`	Active quiz-taking environment. Handles multi-choice selection, identification textfields, passage display, timers, score computation, and answer explanations.
`lib/pages/quiz_folder_page.dart`	`QuizFolderPage`	Hierarchical folder view grouping related quizzes under comprehensive curriculum modules.
`lib/pages/lesson_page.dart`	`LessonPage`	Standard lesson reader presenting markdown content, audio pronunciation buttons, and companion quiz quick-navigation.
`lib/pages/lesson_folder_page.dart`	`LessonFolderPage`	Folder-based curriculum navigator allowing learners to browse lesson sequences by topic and mastery level.
`lib/pages/content_viewer_page.dart`	`ContentViewerPage`	Universal document viewer rendering educational articles, attached PDFs, and rich text resources.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/pages/browse_educators_tab.dart`	`BrowseEducatorsTab`	Educator discovery directory. Displays certified educator cards, student reviews, ratings, subscription tiers, and direct mentorship booking buttons.
`lib/pages/educator_profile_page.dart`	`EducatorProfilePage`	Public educator portfolio showcasing biography, credentials, subscriber counts, published content library, and student ratings.
`lib/pages/manage_subscriptions_page.dart`	`ManageSubscriptionsPage`	Dual-purpose subscription manager: learners manage active educator tiers; educators configure Standard ($10) and Premium ($30) pricing.
`lib/pages/conference_page.dart`	`ConferencePage`	Live Agora RTC video conference portal. Provides bi-directional audio/video streaming, camera flip, mute controls, and synchronized Firestore text chat.
`lib/pages/role_application_page.dart`	`RoleApplicationPage`	Application submission portal for users applying to become certified Educators or Validators, including credential document attachments.
`lib/pages/ai_assessment_generator_page.dart`	`AIAssessmentGeneratorPage`	Specialized standalone interface for educators to rapidly generate comprehensive assessments from raw prompt specifications.

4.5 Educational Notebook Workspaces (`lib/pages/notebook/`)

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/pages/notebook/notebook_list_page.dart`	`NotebookListPage`	Workspace hub displaying all educator-authored notebooks. Supports creating notebooks (`Notebook Title *`) and quick search filtering.
`lib/pages/notebook/notebook_detail_page.dart`	`NotebookDetailPage`	Main notebook workspace. Houses source management drawers, synthesis triggers, output tab bars, and title rename modals.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/pages/notebook/lesson_view.dart`	`LessonView`	Interactive lesson output viewer. Supports inline markdown editing (`Lesson Title *`), dynamic image block insertion, and companion quiz toolbar linking.
`lib/pages/notebook/flashcard_study_page.dart`	`FlashcardStudyPage`	Interactive 3D card review session. Users reveal answers and rate recall quality via 4 SM-2 buttons (Again, Hard, Good, Easy).
`lib/pages/notebook/flashcard_deck_viewer.dart`	`FlashcardDeckViewer`	Comprehensive deck browser displaying all flashcards, individual ease factors, repetition counts, and overall deck mastery percentages.
`lib/pages/notebook/learner_decks_page.dart`	`LearnerDecksPage`	Learner repository of saved flashcard decks showing due review counts and daily spaced repetition progress indicators.
`lib/pages/notebook/study_guide_view.dart`	`StudyGuideView`	Renders synthesized study guides featuring bulleted key takeaways, core vocabulary definitions, and practical usage rules.
`lib/pages/notebook/mind_map_view.dart`	`MindMapView`	Visual concept map viewer rendering hierarchical node-link trees to illustrate grammatical taxonomy and sentence structures.
`lib/pages/notebook/timeline_view.dart`	`TimelineView`	Chronological timeline viewer displaying historical language development milestones or progressive grammatical mastering sequences.
`lib/pages/notebook/faq_view.dart`	`FaqView`	Accordion-style FAQ viewer answering common linguistic questions derived automatically from the ingested source materials.
`lib/pages/notebook/briefing_view.dart`	`BriefingView`	Executive briefing viewer providing quick high-level pedagogical summaries of complex multi-chapter curriculum sources.
`lib/pages/notebook/publish_content_dialogs.dart`	`PublishLessonDialog` , `PublishQuizDialog` , `PublishFlashcardsDialog`	Multi-format publishing modal wizards allowing educators to release synthesized content to the public or subscriber repository with visibility controls.

4.6 Administration & Verification Tabs (`lib/pages/admin/`)

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/pages/admin_dashboard.dart`	`AdminDashboard`	Master administration scaffold with responsive sidebar navigation, telemetry overview, and tab routing across all administrative consoles.
`lib/pages/admin/admin_users_tab.dart`	`AdminUsersTab`	User management console. Allows searching user accounts, changing roles, suspending abusive accounts, and reviewing registration timestamps.
`lib/pages/admin/admin_lessons_tab.dart`	`AdminLessonsTab`	Direct lesson creation and editing interface featuring markdown editor, image dropzone, prompt engineer triggers, and visibility settings.
`lib/pages/admin/admin_quizzes_tab.dart`	`AdminQuizzesTab`	Comprehensive quiz authoring workspace. Configures title (`Title *`), duration (`Duration *`), max attempts (`Max Attempts *`), and multi-type question items.
`lib/pages/admin/admin_assessments_tab.dart`	`AdminAssessmentsTab`	Summative assessment builder with strict attempt caps, time limits, passage questions, and automatic grading schema configurations.
`lib/pages/admin/admin_spelling_words_tab.dart`	`AdminSpellingWordsTab`	Spelling Bee database manager. Allows adding, editing, and deleting vocabulary items with difficulty tiers (`Word *` , `Difficulty *`) and audio links.
`lib/pages/admin/admin_validation_tab.dart`	`AdminValidationTab`	Administrative oversight tab monitoring validator actions, reviewing pending queues, and inspecting content rejection logs.
`lib/pages/admin/validator_dashboard_tab.dart`	`ValidatorDashboardTab`	Dedicated quality assurance console for Validators. Inspects educator-submitted content, reviews grammar accuracy, and issues approvals or audited denials.
`lib/pages/admin/educator_dashboard_tab.dart`	`EducatorDashboardTab`	Educator telemetry console displaying subscriber counts, total lesson views, average ratings, and mentorship schedule overviews.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/pages/admin/educator_groups_tab.dart`	`EducatorGroupsTab`	Live mentorship scheduler. Allows educators to book upcoming Agora video meetings with subscribers (`Meeting Title *` , `Select Student *`).
`lib/pages/admin/admin_notifications_tab.dart`	`AdminNotificationsTab`	Broadcast center allowing administrators to compose and dispatch system-wide push notifications and platform updates.
`lib/pages/admin/admin_landing_settings_tab.dart`	`AdminLandingSettingsTab`	Dynamic CMS tab allowing administrators to modify landing page hero copy, feature highlights, and student testimonials without recompiling.
`lib/pages/admin/admin_lesson_database_tab.dart`	`AdminLessonDatabaseTab`	Comprehensive database view of all lessons across all educators, enabling global search, category tagging, and batch archival.
`lib/pages/admin/lessons_list_tab.dart`	`LessonsListTab`	Compact searchable list view of existing lessons for quick selection and embedding into curriculum modules.

4.7 Reusable UI Components & Widgets (`lib/widgets/`)

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/widgets/ai_quota_badge.dart`	`AIQuotaBadge`	Real-time badge widget displaying remaining daily AI generations with role indicators (e.g. `AI: 45 left`).
`lib/widgets/ai_quota_exceeded_dialog.dart`	`AIQuotaExceededDialog`	Modal displayed when a user exhausts their daily quota, explaining tier boundaries and reset schedules.
`lib/widgets/ai_quota_notification.dart`	`AIQuotaNotification`	Banner notification alerting users when their daily generation limit is approaching exhaustion.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/widgets/content_denial_dialog.dart`	`ContentDenialDialog`	Audited moderation modal requiring a minimum 30-character comprehensive explanation (`DETAILED EXPLANATION *`) before rejecting content.
`lib/widgets/generation_loading_overlay.dart`	`GenerationLoadingOverlay`	Full-screen blocking overlay with animated progress spinner and rotating pedagogical text, preventing double-clicks during AI operations.
`lib/widgets/paymongo_qr_dialog.dart`	`PaymongoQrDialog`	High-contrast QR Ph payment dialog rendering dynamic QR strings with banking instructions, real-time payment status polling, and a single-shot mutex (`_hasDispatchedSuccess`) preventing race conditions between timers and manual confirms.
`lib/widgets/interactive_markdown.dart`	`InteractiveMarkdown`	Styled markdown renderer supporting code syntax highlighting, custom blockquotes, inline audio triggers, and image tap zoom.
`lib/widgets/lesson_image_dropzone.dart`	`LessonImageDropzone`	Interactive image dropzone for lessons supporting drag-and-drop file upload and AI image prompt generation.
`lib/widgets/linked_companion_toolbar.dart`	`LinkedCompanionToolbar`	Sticky toolbar displayed inside lessons that allows learners to directly jump to companion practice quizzes and flashcard decks.
`lib/widgets/custom_app_bar.dart`	`CustomAppBar`	Top bar containing app logo, navigation breadcrumbs, notification badge bell, profile avatar, and theme toggle button.
`lib/widgets/admin_sidebar.dart`	`AdminSidebar`	Collapsible side navigation rail used across admin and educator consoles with active route highlights.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/widgets/app_search_bar.dart`	`AppSearchBar`	Standardized debounced search input field used across catalog, user, and lesson management views.
`lib/widgets/audio_player_widget.dart`	`AudioPlayerWidget`	Compact inline audio player for vocabulary listening drills and pronunciation reference sounds.
`lib/widgets/subscription_tier_dialog.dart`	`SubscriptionTierDialog`	Comparison modal detailing Standard vs Premium subscription benefits and initiating PayMongo checkout.
`lib/widgets/terms_and_conditions_dialog.dart`	`TermsAndConditionsDialog`	Scrollable legal modal rendering privacy policy and terms of service with acceptance verification.
`lib/widgets/user_visibility_selector.dart`	`UserVisibilitySelector`	Segmented button allowing educators to toggle content visibility between Public, Subscribers Only, and Private.
`lib/widgets/video_player_modal.dart`	`VideoPlayerModal`	Modal media player for video lectures and demonstration clips.
`lib/widgets/modern_bottom_nav.dart`	`ModernBottomNav`	Floating pill bottom navigation bar for mobile devices providing fluid route transitions.
`lib/widgets/universal_drawer.dart`	`UniversalDrawer`	Slide-out mobile navigation drawer exposing user profile info, route links, and role switching shortcuts.
`lib/widgets/author_name_widget.dart`	`AuthorNameWidget`	Chip displaying author avatar, display name, and verified Educator/Validator badge.
`lib/widgets/design_ornaments.dart`	`DesignOrnaments`	Decorative background gradient blobs and floating shapes establishing Grammatica's visual aesthetic.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/widgets/markdown_guide_dialog.dart`	`MarkdownGuideDialog`	Reference modal explaining markdown syntax (headings, bold, lists, tables) to educators authoring lessons.
`lib/widgets/application_preview_widgets.dart`	`ApplicationPreviewWidgets`	Renders submitted educator resume documents and teaching certificates inside validator review modals.
`lib/widgets/notification_widgets.dart`	`NotificationTile`	List tile rendering individual notification items with timestamp formatting and mark-as-read toggles.
`lib/widgets/google_sign_in_button.dart` (and `_web`, `_stub`)	`GoogleSignInButton`	Platform-safe branded Google authentication button triggering OAuth workflows across mobile and web.
`lib/widgets/preview_helper.dart` (and `_web`, `_stub`)	`PreviewHelper`	Cross-platform helper resolving local file bytes vs web URLs for previewing images and documents.
`lib/widgets/notebook/flashcard_widget.dart`	`FlashcardWidget`	Animated 3D flipping card widget delivering tactile physics during spaced repetition review sessions.
`lib/widgets/notebook/notebook_card.dart`	`NotebookCard`	Dashboard tile presenting notebook summary metrics, source counts, and last modified timestamps.
`lib/widgets/notebook/source_upload_panel.dart`	`SourceUploadPanel`	Tabbed modal panel for adding sources: PDF file upload, web URL (`Webpage URL *`), or raw note (`Content *`).
`lib/widgets/notebook/output_toolbar.dart`	`OutputToolbar`	Action bar attached to notebook outputs providing export, regenerate, edit, and publish buttons.
`lib/widgets/notebook/generation_confirm_dialog.dart`	`GenerationConfirmDialog`	Confirmation modal informing educators of expected AI generation quota consumption before dispatching jobs.

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/widgets/notebook/notebook_selector_dialog.dart`	`NotebookSelectorDialog`	Selection dialog allowing educators to assign generated content to existing notebooks or initialize new ones.

4.8 Utilities & Platform Helpers (`lib/utils/`)

File Path	Key Classes / Symbols	Architectural Responsibility & Implementation Details
`lib/utils/responsive_layout.dart`	`ResponsiveLayout`	Provides responsive breakpoint utilities (Mobile < 768px, Tablet 768-1100px, Desktop > 1100px) and conditional builders.
`lib/utils/image_utils.dart`	`ImageUtils`	Image utility functions for resizing, compressing, base64 encoding/decoding, and format validation prior to cloud upload.
`lib/utils/file_helper.dart` (and `_io` , `_stub`)	`FileHelper`	Cross-platform abstraction for downloading, reading, and writing files to local disk storage across desktop, mobile, and web.

5.0 Microservices AI Backend & LLM Fine-Tuning

While the Flutter client directly connects to Firebase for primary application data, computationally intensive and secured operations are delegated to Grammatica's Python microservices layer:

5.1 FastAPI Microservice Architecture (`ai_backend/`)

- **REST API Gateway (`main.py`):** High-performance asynchronous API built on FastAPI and Uvicorn, secured by JWT and Firebase ID Token verification middleware (`middleware/auth.py`).
- **Distributed Idempotency Engine (`middleware/idempotency.py` & `services/idempotency_store.py`):** Intercepts mutating API calls, validates `Idempotency-Key` headers, enforces async locks, calculates SHA-256 payload fingerprints, returns `HTTP 409 Conflict` on in-flight collisions, rejects mutations via `HTTP 422` , and serves replay-cached responses with `Idempotent-Replay: true` .
- **Generation Routes (`routes/lesson.py` , `routes/quiz.py` , `routes/notebook.py`):** Endpoints accepting ingested curricular sources and calling Google Gemini 2.5 Flash with strict JSON schema constraints.
- **Payment & Webhook Listener (`routes/payment.py`):** Receives PayMongo webhook callbacks, verifies cryptographic signature headers, locks on webhook event IDs (`evt_...`) for deduplication, and updates user subscription records in Firestore.
- **Email Dispatcher (`routes/email.py`):** Dispatches transactional receipts and notifications using the Brevo API.
- **Document Processing (`services/pdf_service.py`):** Uses `pdfplumber` and `pypdf` to parse complex multi-page academic documents, stripping non-textual artifacts and chunking text to prevent token overflow.

5.2 LLaMA 1B LoRA Fine-Tuning Pipeline (`backend/`)

In addition to Google Gemini, Grammatica features an internal, domain-specific fine-tuning pipeline for lightweight on-premise execution:

- **Dataset Curation (`lessons_data.csv` & `csv_to_jsonl.py`):** Curated grammar instruction dataset comprising thousands of question-answer pairs, rule explanations, and common grammatical misconception corrections formatted in JSON Lines (`dataset.jsonl`).
- **Parameter-Efficient Fine-Tuning (PEFT / LoRA):** Uses Low-Rank Adaptation (LoRA) on the LLaMA 1B base model (`train_llama_1b.ipynb`), adapting attention projection matrices (W_q, W_v) while keeping base parameters frozen. Adapter weights and tokenizer configurations are preserved in `backend/model/lora/` .

6.0 Top 27 Mock Defense Questions & Comprehensive Panel Answers

This section contains anticipated questions frequently raised by academic panels, complete with authoritative technical answers.

Q1: Why did your team select Flutter instead of separate native Android and React Web codebases? Architecture

Answer: Flutter was selected to achieve **single-codebase parity** across Web and Android APK without compromising on performance. Flutter compiles directly to native ARM machine code on mobile and optimized JavaScript/WebAssembly with Skia canvas rendering on Web. This unified architecture saved approximately 40% of development overhead and guaranteed that complex features—such as the SuperMemo SM-2 spaced repetition engine, interactive markdown parsing, and PayMongo payment flows—behave identically on desktop browsers and mobile handsets.

Q2: How does Grammatica mitigate AI hallucinations in generated lessons and quizzes? AI & Reliability

Answer: We implement a **Grounding-First Architecture**. Rather than issuing zero-shot prompts to the LLM, the educator must first ingest verified pedagogical sources (PDFs, syllabus chapters, scholarly web pages). In `notebook_prompts.dart`, the system prompt strictly instructs the model: "*Synthesize strictly and exclusively from the provided source context. Do not invent ungrounded facts or rules.*" Furthermore, every generated output preserves source citation links, and public content must undergo human-in-the-loop audit by certified Validators before publication.

Q3: Can you mathematically explain the SuperMemo SM-2 algorithm implemented in your flashcard system? Cognitive Science

Answer: The SM-2 algorithm models the human forgetting curve. After reviewing a card, the learner assigns a quality score q from 0 to 5. The Ease Factor (EF) is updated as: $EF' = EF + (0.1 - (5 - q) imes (0.08 + (5 - q) imes 0.02))$, with a strict floor of 1.30. If $q < 3$, the card is treated as failed: the repetition counter n resets to 0 and the interval resets to 1 day. If $q \ge 3$, the interval progresses: $I(1) = 1 ext{ day}$, $I(2) = 6 ext{ days}$, and for $n \ge 2$, $I(n) = I(n-1) imes EF'$. This minimizes over-testing mastered concepts while ensuring vulnerable memories are reinforced right before decay.

Q4: How does your 3-Attempt Lockout system protect against brute-force attacks? Security

Answer: `DeviceSessionService` tracks failed authentication attempts. Upon the 3rd consecutive failed login, the device session is locked for 5 minutes. The timestamp is stored with cryptographic session IDs in local storage and mirrored against Firestore login telemetry. Furthermore, we implement a 15-minute sliding window: if an attacker makes 2 failed attempts and pauses, the failure counter resets after 15 minutes of inactivity to avoid permanently penalizing legitimate users who mistyped their credentials.

Q5: What happens when a user deletes their account? How do you handle data privacy and orphaned records? Security & Privacy

Answer: In compliance with GDPR and data privacy standards, account deletion executes a cascading purge in `DatabaseService.deleteAccount()`. All personal identity records, active subscriptions, private flashcard decks, and session logs are wiped. However, published lessons authored by Educators remain in the repository marked as "*Anonymous Educator*" so that enrolled learners do not suffer sudden curriculum deletion. Sensitive re-authentication is mandatory before deletion can proceed.

Q6: What is the role of a Validator, and how do you prevent arbitrary content rejection?

Governance & Moderation

Answer: Validators are certified pedagogical moderators who review community submissions. To eliminate arbitrary or retaliatory rejections, `ContentDenialDialog` enforces a **mandatory 30-character detailed explanation** requirement. A Validator cannot reject content without documenting specific grammatical errors or policy violations. This explanation is permanently logged in Firestore and delivered to the educator, creating full transparency and an audit trail for administrators.

Q7: How does Agora RTC video conferencing work across Web and Android APK without crashes?

Video & WebRTC

Answer: `AgoraService` initializes the Agora RTC client with platform-specific video rendering strategies. On native Android APK, video streams bind to native hardware SurfaceViews. On Web, the client attaches to HTML5 canvas/video DOM elements. Dynamic token authentication is calculated via HMAC-SHA256 credentials to ensure rooms cannot be accessed by unauthorized callers. Concurrently, a Firestore subcollection powers an synchronized real-time text chat alongside the call.

Q8: How do you process payments and handle localized payment methods in the Philippines?

Monetization

Answer: We integrated the **PayMongo API**, the leading FinTech payment gateway in the Philippines. It natively supports GCash, Maya, GrabPay, and credit/debit cards. We implement two complementary flows: (1) PayMongo Hosted Checkout Sessions for comprehensive browser payments, and (2) Direct QR Ph code generation (`PaymongoQrDialog`), allowing users to scan national QR codes directly from any Philippine e-wallet or banking app.

Q9: What happens if an external email delivery API goes down during payment confirmation?

Backend & Email

Answer: `EmailSenderService` implements a **Dual-Relay Resilience Architecture**. The primary dispatcher uses the Brevo (Sendinblue) transactional REST API. If Brevo fails or encounters network timeout, the request gracefully falls back to our local Dart email relay script (`scripts/local_email_relay.dart`) or queues the receipt event in Firestore for asynchronous background retries, ensuring the user's payment record is never lost.

Q10: Why did you incorporate the Vosk Speech Model for pronunciation quizzes instead of relying exclusively on cloud speech APIs?

Speech Recognition

Answer: Cloud speech-to-text APIs introduce high latency, require constant internet connectivity, and impose recurring per-minute API costs. By compiling the **Vosk Acoustic Model** into the native Android APK via C++ platform channels (`vosk_service_native.dart`), speech decoding occurs entirely on the user's mobile device with zero latency and complete privacy. On the web platform, we conditionally switch to the Web Speech Recognition API.

Q11: How does the platform protect against API cost explosions from excessive AI generations?

Rate Limiting & Cost

Answer: `AIRateLimitService` enforces strict daily quotas calibrated by role: Learners receive 20 requests/day, Educators receive 50 requests/day, Validators receive 100 requests/day, while Admins enjoy unlimited access. Counters are tracked in Firestore user quota documents and verified server-side. Additionally, `GenerationLoadingOverlay` blocks redundant double-clicks while AI jobs are in flight.

Q12: Why Cloud Firestore? What are its strengths and how do you handle indexing?

Database Design

Answer: Cloud Firestore provides high-performance, document-oriented NoSQL persistence with native real-time synchronization (`StreamSubscription`). This real-time capability is essential for live in-call chat, instant notification badges, and real-time validation queue updates. Complex compound queries (e.g. querying lessons filtered by educator ID, visibility tier, and sorted by creation timestamp) are optimized via pre-compiled composite indexes defined in `firestore.indexes.json` .

Q13: What state management approach is used across the Flutter frontend?

Architecture & State

Answer: We combine the **Repository/Service Pattern** with Flutter's native reactive primitives (`StatefulWidget` , `StreamBuilder` , and `ValueNotifier`). State is encapsulated inside dedicated singletons (e.g., `AuthService` , `RoleService` , `SpacedRepetitionService`) that expose broadcast streams. This eliminates boilerplate while avoiding unnecessary rebuilds across distant widget trees.

Q14: How does Grammatica support different learning modalities (Visual, Auditory, Reading, Kinesthetic)?

Educational Pedagogy

Answer: Grammatica embraces the VARK pedagogical framework:

- **Visual:** Hierarchical Mind Maps (`MindMapView`) and Chronological Timelines (`TimelineView`).
- **Auditory:** Audio player drills, native Vosk pronunciation scoring, and live Agora video calls.
- **Reading/Writing:** In-depth interactive markdown lessons, executive briefings, and comprehensive study guides.
- **Kinesthetic:** Gamified Spelling Bee letter-typing, tactile 3D flashcard flipping, and interactive multi-choice quizzes.

Q15: How do you handle CORS issues when running the Flutter Web application?

Cross-Platform Engineering

Answer: Web browsers strictly enforce the Same-Origin Policy (SOP). Direct client-side HTTP calls to third-party domains (such as scraping educational websites or querying raw APIs) are blocked by CORS. We solved this in `WebServiceWeb` by routing web HTTP requests through our dedicated FastAPI backend proxy (`ai_backend`), which executes the HTTP handshake server-to-server and attaches the requisite `Access-Control-Allow-Origin: *` headers before returning data to the web SPA.

Q16: What prompt engineering strategies ensure that Gemini outputs valid JSON?

AI Engineering

Answer: In `ai_logic_service.dart` , we pass `response_mime_type: "application/json"` in the Gemini generation configuration. Furthermore, we define explicit Pydantic-style JSON schema specifications in the system prompt, forbidding markdown code fences (```json) inside raw API returns. The response is validated through try-catch JSON decoders with fallback sanitize regexes to eliminate trailing commas or incomplete brackets.

Q17: How do you test the system? What test suites exist in the codebase?

Testing & Quality Assurance

Answer: The project maintains an automated test suite comprising **63+ comprehensive test cases**. This includes unit tests for SM-2 interval calculations (`spaced_repetition_test.dart`), brute-force lockout window tests (`lockout_system_test.dart`), Agora token generation tests (`agora_service_env_test.dart`), model serialization tests (`study_guide_models_test.dart`), widget tests for dropzones and quota badges (`ai_rate_limit_visual_test.dart`), end-to-end distributed idempotency tests (`idempotency_system_test.dart`), and backend FastAPI replay/conflict test suites (`test_idempotency.py`). All automated tests pass cleanly in CI.

Q18: Why is there a separate Validator role? Why not let Admins perform validation?

Role Separation

Answer: Separation of Concerns. Administrators handle infrastructure, server configurations, landing page settings, and financial reconciliations. Validators are domain specialists (linguists, certified English teachers) whose sole responsibility is verifying grammatical accuracy and curricular standards. Combining these roles would introduce operational bottlenecks and violate the Principle of Least Privilege.

Q19: How does Grammatica ensure accessibility and clear user feedback across complex forms? UI/UX & Accessibility

Answer: Every required field across the application is explicitly designated with an asterisk (`*`) in its label or hint text. Inline regex validators provide instantaneous feedback (e.g., password matching, email structure, min 6 characters). High-contrast dark and light themes adhere to WCAG 2.1 color contrast standards, and responsive wrappers ensure legible layouts on screens ranging from small smartphones to 4K displays.

Q20: How does the system scale if user enrollment jumps from 100 to 100,000? Scalability

Answer: Grammatica scales horizontally at every tier:

- **Client:** Static assets are distributed via Firebase Hosting's global CDN edge network.
- **Database:** Cloud Firestore scales automatically to handle millions of concurrent connections with automatic sharding.
- **AI Microservices:** The stateless FastAPI gateway can be deployed in containerized auto-scaling clusters (e.g. Google Cloud Run or AWS ECS).
- **Video:** Agora RTC offloads all media encoding and packet routing to its global Software-Defined Real-Time Network (SD-RTN).

Q21: What is the difference between your Gemini pipeline and the LLaMA LoRA model in `backend/` ? Fine-Tuning vs RAG

Answer: Gemini 2.5 Flash operates via **Retrieval-Augmented Generation (RAG)**—it ingests dynamic, user-provided source documents at inference time to generate multi-format pedagogical artifacts. The fine-tuned LLaMA 1B LoRA model represents a specialized research prototype trained on specific grammatical error correction datasets, demonstrating how Grammatica can execute on-premise, privacy-sensitive grammar classification without third-party API dependencies.

Q22: What features of Grammatica function when the user loses internet access? Offline Capabilities

Answer: (1) The native Android APK executes speech pronunciation scoring offline using the embedded Vosk acoustic model. (2) Firestore enables offline persistence, caching downloaded lessons and flashcard decks so learners can continue review sessions in offline environments. Mutations (such as SM-2 card quality updates) queue locally and synchronize upon network reconnection.

Q23: What are the current limitations of Grammatica, and how will future iterations address them? Future Work

Answer: Current limitations include reliance on third-party cloud LLM availability and manual appointment booking for mentorship. Future roadmaps incorporate: (1) Automated schedule conflict detection via Google Calendar integration, (2) Real-time conversational AI voice agents for 24/7 conversational immersion practice, and (3) Collaborative multi-student study rooms within the Agora video portal.

Q24: How are complex mind maps and timelines represented in Firestore? Data Structure

Answer: In `study_guide_models.dart` , mind maps are structured as recursive adjacency lists: each `MindMapNode` contains an identifier, label, optional explanation, and a list of nested child nodes. Timelines are represented as serialized arrays of `TimelineEvent` objects containing dates, titles, and descriptions, sorted chronologically prior to UI rendering.

Q25: What is the primary academic contribution of this capstone project?

Academic Significance

Answer: The primary academic contribution is the **synthesis of generative AI document grounding with cognitive spaced retrieval and live synchronous mentorship**. While existing platforms either provide static drills or unguided AI chatbots, Grammatica proves that grounding LLMs in verified curricular sources and coupling them with empirical memory algorithms (SM-2) and human educator verification delivers a significantly more reliable, accountable, and effective digital language learning ecosystem.

Q26: How does Grammatica prevent double billing and duplicate payment records if a learner taps the checkout button multiple times or suffers network drops during payment?

Financial Architecture & Idempotency

Answer: We enforce an end-to-end **3-Tier Distributed Idempotency Pipeline**:

- **Client Tier:** `IdempotencyService` employs *single-flight coalescing* (`executeDeduplicated`). If a user taps "Pay" multiple times rapidly, subsequent taps await the existing in-flight Future rather than firing duplicate HTTP requests. It also computes deterministic keys scoped to 1-minute windows (e.g. `pay_link_{studentId}_{educatorId}_{timestampMinutes}`).
- **API Gateway Tier:** `IdempotencyMiddleware` forwards the key to PayMongo and checks our FastAPI `IdempotencyStore` . If a concurrent duplicate arrives, the server locks on the key and immediately returns `HTTP 409 Conflict` (with `Retry-After: 2`). When completed, repeated calls replay the cached JSON with an `Idempotent-Replay: true` header.
- **Database Ledger Tier:** In Firestore, payments use deterministic document IDs: `/payments/pay_{linkId}` and register in `/idempotency_keys/{key}` . In `DatabaseService.recordPayment()` , a check-and-set transaction inspects if the record is already `paid` . If so, it returns `PaymentRecordResult(isDuplicate: true)` , bypassing duplicate subscription extensions, notification broadcasts, and duplicate Brevo email receipts.

Q27: How do you handle PayMongo webhook at-least-once delivery and race conditions with client-side polling?

Distributed Systems

Answer: Payment gateways guarantee *at-least-once* webhook delivery, meaning webhooks may arrive multiple times, often concurrently with the client's foreground QR polling loop (`PaymongoQrDialog`). We resolve this race condition through two synchronized guards:

1. **Webhook Mutex:** In `routes/payment.py` , the FastAPI webhook listener locks on the PayMongo event identifier (`evt_...`) via `IdempotencyStore` , rejecting duplicate webhook retries.
2. **UI Single-Shot Lock & Transactional Check-and-Set:** `PaymongoQrDialog` maintains a private boolean lock `_hasDispatchedSuccess` . Whichever path finishes first—the webhook or client poll—commits the record to Firestore. The secondary caller checks the deterministic record ID, recognizes the `status == 'paid'` state, flags `isDuplicate: true` , and gracefully exits without side effects.

Grammatica Capstone Defense Reviewer • Compiled for Academic Evaluation • All Rights Reserved © 2026